Comparative Analysis of Execution Efficiency and System Accessibility in Native and Browser-Based Runtime Environments

Prasham Kumbhare
B.Tech Computer Science Engineering
DY Patil International University
India

***Abstract*—Modern software systems are developed using both native compiled languages and browser-based technologies, each offering different advantages in terms of performance, accessibility, portability, and security. This study presents a comparative analysis between native compiled execution environments represented by the C programming language and managed/browser-based runtime environments represented by Python and JavaScript. Experimental benchmarks were conducted to evaluate execution efficiency using large-scale iterative computations and filesystem accessibility operations. The results demonstrated that native compiled execution in C significantly outperformed interpreted execution in Python in computation-intensive tasks. Additionally, native programs were observed to possess direct operating system and filesystem interaction capabilities, whereas browser-based JavaScript execution remained restricted by sandbox security mechanisms and permission-based access control. The study highlights the trade-offs between execution efficiency, hardware accessibility, abstraction, portability, and security in modern computing environments.**

I. Introduction

Modern computing systems utilize multiple execution environments ranging from low-level native compiled languages to high-level browser-based technologies. Native programming languages such as C provide direct interaction with hardware resources and operating system services, enabling high execution efficiency and low-level system control.

In contrast, browser-based technologies such as JavaScript operate within sandboxed environments designed to prioritize security, portability, and user protection.

Over the past decades, browser technologies have evolved significantly and are now capable of executing increasingly complex applications. However, despite advancements in runtime optimization techniques such as Just-In-Time (JIT) compilation and modern JavaScript engines, browser-based environments still exhibit limitations in direct system accessibility and execution efficiency compared to native compiled languages.

This study experimentally compares execution efficiency and filesystem accessibility between native compiled execution and managed/browser-based runtime environments.

II. Research Objectives

The objectives of this study are:

- Compare execution efficiency between native compiled and interpreted runtime environments
- Analyze filesystem accessibility differences
- Investigate runtime abstraction overhead
- Examine browser sandbox restrictions
- Study security versus accessibility trade-offs

III. Methodology

The experiments were conducted on a Windows-based computing environment using C, Python, and browser-based JavaScript.

The first experiment involved iterative summation operations over 100 million iterations to evaluate execution efficiency. Equivalent computational logic was implemented in both C and Python to ensure fairness and consistency in workload comparison.

The second experiment analyzed filesystem accessibility by comparing direct file operations in C with browser-mediated file access mechanisms in JavaScript.

Experimental observations were analyzed comparatively to identify differences in runtime architecture, execution models, security restrictions, abstraction overhead, and system-level accessibility.

IV. Literature Review

Previous studies have explored the relationship between runtime abstraction, execution efficiency, and system accessibility in modern programming environments.

Research on interpreter performance has shown that high-level interpreted languages experience performance overhead due to dynamic typing, runtime object management, and abstraction layers. Studies analyzing Python interpreter behavior demonstrated that interpreted execution environments introduce additional computational overhead compared to native compiled execution models.

Several studies have also investigated browser sandbox architectures and JavaScript security models. Modern browsers implement strict sandboxing mechanisms to isolate web applications from direct hardware and filesystem access. These restrictions are designed to protect user privacy, prevent malicious code execution, and reduce operating system vulnerabilities.

Research involving JavaScript runtime optimization has highlighted the role of modern JavaScript engines and Just-In-Time (JIT) compilation techniques in improving browser execution performance. Despite such optimizations, browser-based environments still operate through additional abstraction layers compared to native compiled execution.

V. Experimental Results

A. Execution Efficiency Benchmark

A computational benchmark involving iterative summation over 100 million iterations was performed.

TABLE I
Execution Time Benchmark

Language	Operation	Time (seconds)
C	100 million iterations	0.046
Python	100 million iterations	3.73

The benchmark demonstrated that Python execution required approximately:

$$\frac{3.73}{0.046} \approx 81$$

times more execution time compared to native compiled C execution for the same computational workload.

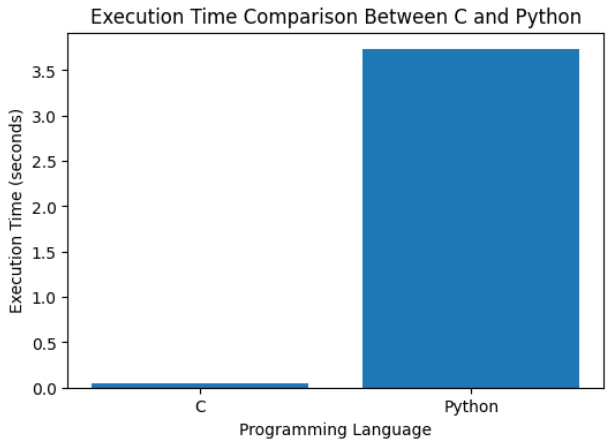

Fig. 1. Execution time comparison between C and Python

The observed performance difference highlights the influence of runtime abstraction, interpreter overhead, and dynamic object management in high-level execution environments.

The observed performance difference can be attributed to several architectural and runtime characteristics:

- Native machine-level compilation
- Static typing in C
- Reduced runtime overhead
- Dynamic object management in Python
- Interpreter and abstraction overhead

B. Filesystem Accessibility Analysis

The native C implementation successfully:

- Created files directly
- Wrote data into files
- Read data from files
- Accessed the operating system filesystem directly

In contrast, browser-based JavaScript execution required explicit user interaction and permission-based file selection dialogs.

TABLE II
Accessibility Comparison

Feature	Native C	Browser JavaScript
Direct filesystem access	Yes	Restricted
Sandbox restrictions	Minimal	Strong
Permission required	No	Yes
Hardware accessibility	Higher	Limited
Security isolation	Lower	Higher

The experiment demonstrated that browser execution environments intentionally restrict low-level system accessibility to improve security and user protection.

VI. Discussion

The experimental findings indicate a strong relationship between execution model design and system-level capabilities.

Native compiled execution environments prioritize:

- High computational efficiency
- Low-level system accessibility
- Minimal abstraction overhead

Browser-based and managed runtime environments prioritize:

- Security isolation
- Portability
- Controlled resource access
- Runtime flexibility

The study highlights an important architectural trade-off in modern computing systems:

- Native environments provide higher performance and deeper system accessibility
- Browser-based environments provide stronger security isolation and portability

Browser sandbox security mechanisms intentionally restrict arbitrary filesystem access to protect users from malicious web applications.

VII. Conclusion

This study presented a comparative analysis of execution efficiency and system accessibility between native compiled and browser-based runtime environments.

Experimental benchmarking demonstrated that native compiled execution using C significantly outperformed interpreted execution using Python in computation-intensive workloads.

Additionally, filesystem accessibility experiments demonstrated that native execution environments possess deeper operating system interaction capabilities, whereas browser-based environments operate under sandbox security restrictions and permission-based access models.

The study identified runtime abstraction, dynamic typing, interpreter overhead, and browser sandboxing as major factors influencing performance and accessibility differences.

VIII. Limitations

This study utilized simplified benchmark experiments within limited runtime scenarios. Advanced runtime behaviors such as JIT optimization variability, multithreading, memory fragmentation, compiler optimization levels, and browser engine implementation differences were not extensively analyzed.

IX. Future Work

Future research may investigate:

- WebAssembly runtime performance
- Node.js execution environments
- Rust and memory-safe runtime systems
- Memory utilization benchmarking
- CPU utilization analysis
- Hybrid runtime architectures

Further studies may also evaluate security-performance trade-offs in hybrid runtime environments and modern browser optimization architectures.

References

[1] Gergö Barany, "Python Interpreter Performance Deconstructed," ACM Digital Library.

[2] Yinzhi Cao et al., "Virtual Browser: Sandbox Third-party JavaScripts."

[3] Marija Selakovic and Michael Pradel, "Performance Issues and Optimizations in JavaScript."

[4] B. Kernighan and D. Ritchie, *The C Programming Language*, 2nd ed., Pearson Education.

[5] Mozilla Developer Network (MDN), "JavaScript Execution Model."